

CSE 344 - System Programming

Homework #4: Concurrent Log Analysis Pipeline

Student Name: Ömer Zelka

Student ID: 230104004020

University: Gebze Technical University

Date: May 2, 2026

Important Note to the Grader

Due to the strict assignment requirement that all terminal screenshots must be **"uncut"** (showing the full execution from command prompt through program exit), the resulting image files are extremely long vertically. Attempting to embed these images directly into the PDF causes a fatal `Dimension too large` LaTeX compilation error because they exceed the maximum allowable page dimensions.

To strictly adhere to the "uncut" rule and preserve image readability, **all required test screenshots (Test 1 to 7) and the Architecture Diagram are provided as separate, full-resolution image files in the submitted ZIP archive.** Please review these files alongside this report.

Contents

1	Introduction	2
2	System Architecture	2
3	Shared Memory Layout	2
3.1	Region A: Dispatcher Input Queue	2
3.2	Region B: Per-Level Analysis Buffers	2
3.3	Region C: Results Area	2
3.4	Region D: High-Priority Buffer	2
4	Lock Ordering and Deadlock Prevention	3
5	Thread Lifecycle	3
6	TLS Design	3
7	Dispatcher Logic	3
8	Termination Chain	3
9	Test Scenarios	3
10	Challenges and Solutions	4

1 Introduction

The objective of this project is to design and implement a multi-process and multi-threaded Concurrent Log Analysis Pipeline in C on a POSIX-compatible system. The system reads multiple log files, classifies log entries by severity, and identifies high-priority sources using a centralized dispatcher.

The main challenges of this assignment include the orchestration of seven distinct process and thread types, the correct synchronization of four shared memory regions using various primitives (mutexes, condition variables, semaphores, and barriers), and the implementation of thread-local storage (TLS) for efficient per-thread scoring.

2 System Architecture

The system architecture consists of seven interacting components: Parent Coordinator, Reader Processes, Dispatcher Process, Analyzer Processes (x4), Aggregator Process, and a Watchdog Thread.

Note to Grader: The Architecture Diagram is provided as a separate file. Please click the link below to view it:

[View Architecture Diagram (architecture_diagram.png)]

3 Shared Memory Layout

The system utilizes four shared memory regions (A, B, C, D) mapped via `mmap (MAP_SHARED | MAP_ANONYMOUS)`.

3.1 Region A: Dispatcher Input Queue

Protected by `input_mutex` and condition variables (`not_full_a`, `not_empty_a`) with `PTHREAD_PROCESS_SHARED` attributes.

3.2 Region B: Per-Level Analysis Buffers

Four independent buffers (one per level). Written by Dispatcher, read by specific Analyzer.

3.3 Region C: Results Area

Contains `level_result_t` structures and semaphores for Aggregator notification.

```
1 typedef struct {
2     char level[8];
3     long total_entries;
4     double total_weighted_score;
5     double per_keyword_score[MAX_KEYWORDS];
6     double per_thread_score[MAX_WORKERS];
7     char top_source[3][64];
8     long top_source_hits[3];
9     int ready;
10 } level_result_t;
```

3.4 Region D: High-Priority Buffer

Stores entries from filtered sources. Protected by `priority_mutex`.

4 Lock Ordering and Deadlock Prevention

A strict global lock acquisition order is enforced to prevent circular wait:

Order	Mutex
1st	Region A Mutex
2nd	Region B Level Mutex
3rd	Region C Result Mutex
4th	Region D Priority Mutex

Table 1: Lock Acquisition Order

5 Thread Lifecycle

Reader threads divide files into byte ranges and push parsed entries into internal buffers. Worker threads in Analyzer processes consume from Region B, perform analysis using TLS, and synchronize at a barrier before exiting.

6 TLS Design

Worker threads use `pthread_key_t` for per-thread scoring. Scores are accumulated in a heap-allocated array and flushed to Region C inside the TLS destructor after the thread exits, ensuring data integrity.

7 Dispatcher Logic

The Dispatcher reads from Region A using `pthread_cond_timedwait`. It handles EOF markers by counting them per level and forwarding a final EOF to Region B only when all Reader processes have completed.

8 Termination Chain

The chain starts with Readers exiting after placing EOFs, followed by the Dispatcher, Analyzers (after posting semaphores), and finally the Aggregator. The Parent joins the Watchdog and cleans resources.

9 Test Scenarios

Due to format constraints and to preserve the "uncut" requirement of the terminal screenshots, the images are provided as separate files in the same directory as this report. Please click the links below to view each corresponding test result.

- **Test 1 (Baseline):** [Click to View test1_baseline.png]
Shows baseline execution with a single keyword, human-readable format, and correct binary check.
- **Test 2 (Multi-keyword):** [Click to View test2_multikeyword.png]
Shows per-keyword score breakdown and weighted scoring correctly applied.
- **Test 3 (Single thread):** [Click to View test3_singlethread.png]
Demonstrates sequential correctness, matching totals from Test 1.

- **Test 4 (High concurrency):** [Click to View test4_highconcurrency.png]
Shows the system handles 8 readers and 64 workers with high load deterministically.
- **Test 5 (No priority matches):** [Click to View test5_nopriority.png]
Demonstrates execution without crashing when the filter file is empty.
- **Test 6 (Valgrind):** [Click to View test6_valgrind.png]
Confirms 0 memory leaks and 0 errors.
- **Test 7 (Ctrl+C):** [Click to View test7_ctrlc.png]
Shows no zombie processes or leftover shared memory segments after a manual interruption.

10 Challenges and Solutions

1. **Challenge:** Thread safety in shared memory. **Solution:** Initialized all primitives with `PTHREAD_PROCESS_SHARED`.
2. **Challenge:** Overlapping keyword search. **Solution:** Implemented manual sliding window indexing.
3. **Challenge:** Atomic checkpointing. **Solution:** Used `rename(2)` for the binary output.